

선형대수학 기본

남궁명수

행렬과 벡터의 출발점: 연립일차방정식의 구조

[1] 연립일차방정식에서 출발하기

다음과 같이 2 차 연립일차방정식을 생각해보자.

$$\begin{cases} x + 2y = 4 \\ 2x + 5y = 9 \end{cases}$$

이 식은 미지수 x, y 가 동시에 만족해야 하는 두 개의 선형 조건을 나타낸다.
이러한 구조가 바로 행렬과 벡터 개념이 등장하는 출발점이다.

[2] 행렬과 벡터로의 표현

위 연립방정식은 다음과 같이 행렬과 벡터의 곱으로 표현할 수 있다.

$$\begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 4 \\ 9 \end{bmatrix}$$

여기서 각 요소의 의미는 다음과 같다.

- 왼쪽의 행렬: 계수 행렬(coefficient matrix)
- 가운데 벡터: 미지수 벡터(column vector)
- 오른쪽 벡터: 결과 벡터

이 표현은 여러 개의 방정식을 하나의 구조로 묶어 표현할 수 있게 해준다.

[3] 행 벡터와 열벡터

벡터는 방향에 따라 다음과 같이 구분된다.

- column vector(열 벡터)

$$\begin{bmatrix} x \\ y \end{bmatrix}$$

- row vector(행 벡터)

$$\begin{bmatrix} x & y \end{bmatrix}$$

일반적으로 선형대수와 컴퓨터공학에서는 열 벡터(column vector)를 기준으로 표현하는 경우가 훨씬 많다.

이는 변환(행렬)이 벡터에 작용하는 구조를 자연스럽게 표현할 수 있기 때문이다.

[4] 행렬 곱을 다시 연립방정식으로 이해하기

행렬 곱의 계산은 다음과 같은 원리를 따른다.

- 행렬의 한 행(row)과 벡터의 한 열(column)을 내적한다.

예를 들어,

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 1 \cdot x + 2 \cdot y$$

이는 정확히 첫 번째 방정식

$$x + 2y$$

를 의미한다.

마찬가지로 두 번째 행은

$$2x + 5y$$

를 만들어 낸다.

즉, 행렬 곱은 연립방정식을 압축해 놓은 표현이라고 볼 수 있다.

[5] 더 근본적인 해석: 선형 결합 관점

행렬-벡터 곱은 다음과 같이 선형 결합(linear combination)으로도 해석할 수 있다.

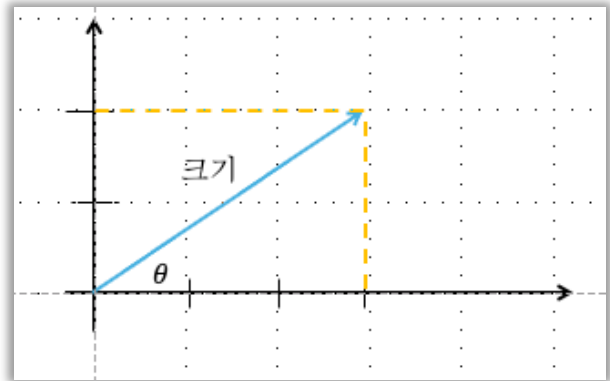
$$x \begin{bmatrix} 1 \\ 2 \end{bmatrix} + y \begin{bmatrix} 2 \\ 5 \end{bmatrix} = \begin{bmatrix} 4 \\ 9 \end{bmatrix}$$

이 관점에서 보면

- 행렬의 각 열(column)은 하나의 벡터이며
- 미지수 x, y 는 그 벡터들을 얼마나 섞을지 결정하는 계수이다.

즉, 행렬×벡터 = 열 벡터 들의 선형 결합 이라는 매우 중요한 해석이 나온다.

벡터의 크기와 방향



[1] 벡터의 크기

2차원 평면에서 벡터

$$v = (x, y)$$

가 주어졌을 때, 벡터의 크기(길이)는 피타고라스 정리에 의해 다음과 같이 정의된다.

$$|v| = \sqrt{x^2 + y^2}$$

예를 들어

$$v = (3, 2)$$

라면,

$$|v| = \sqrt{3^2 + 2^2} = \sqrt{13}$$

이는 벡터를 원점에서 시작하는 직선으로 보았을 때, 그 직선의 실제 길이를 의미한다.

[2] 벡터의 방향

벡터의 방향은 보통 x 축을 기준으로 한 각도 θ 로 표현한다.
이 각도는 삼각함수 중 탄젠트(tan)를 이용해 구할 수 있다.

벡터(x, y)를 직각삼각형으로 해석하면,

- 밑변: x
- 높이: y

이므로,

$$\tan \theta = \frac{y}{x}$$

따라서 각도 θ 는 다음과 같이 계산된다.

$$\theta = \tan^{-1}\left(\frac{y}{x}\right)$$

예를 들어,

$$v = (3, 2)$$

일 때,

$$\theta = \tan^{-1}\left(\frac{2}{3}\right)$$

이 값은 벡터가 x 축에서 얼마나 기울어져 있는지를 나타낸다.

[3] 크기와 방향의 의미 정리

벡터는 단순한 숫자 묶음이 아니라, 다음 두 요소가 결합된 개념이다.

- 크기: 얼마나 멀리 가는가
- 방향: 어느 쪽으로 가는가

즉, 벡터 = 크기×방향 이라는 구조를 가진다.

이 때문에 같은 크기를 가진 벡터라도 방향이 다르면 완전히 다른 벡터가 되며,
반대로 방향이 같더라도 크기가 다르면 다른 의미를 갖는다.

전치 행렬(Transpose Matrix)

[1] 전치 행렬의 정의

행렬 A 가 원소 a_{ij} 로 구성되어 있을 때,
전치 행렬 A^T 는 행과 열을 서로 바꾼 행렬이다.

$$A = (a_{ij}) \Rightarrow A^T = (a_{ji})$$

즉,

- 원래의 i 행 j 열 원소 $\rightarrow$ 전치 후 j 행 i 열 원소

예시

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

[2] 전체 행렬의 기본 성질

[2-1] 전치를 두 번하면 원래 행렬

$$(A^T)^T = A$$

[2-2] 덧셈에 대한 전치

$$(A + B)^T = A^T + B^T$$

행렬 덧셈과 전치는 서로 간섭하지 않는다.

[2-3] 곱셈에 대한 전치

$$(AB)^T = B^T A^T$$

순서가 반대로 바뀐다

특수한 경우

- | |
|---|
| <ul style="list-style-type: none">- $(A^T A)^T = A^T A$- $(A A^T)^T = A A^T$ |
|---|

이러한 형태는 전치해도 자기 자신으로 돌아온다.

[2-4] 스칼라 배에 대한 전치

$$c(A^T) = cA^T$$

스칼라는 행렬 구조에 영향을 주지 않는다.

[2-5] 행렬과 전치

$$\det(A^T) = \det(A)$$

전치의 부피(면적) 정보를 바꾸지 않는다.

[2-6] 역행렬과 전치

$$(A^T)^{-1} = (A^{-1})^T$$

전치와 역행렬 연산은 서로 교환 가능

[3] 대칭 행렬(Symmetrix matrix)

정의: 행렬 A 가 자기 자신의 전치와 같다면, 이를 대칭 행렬이라 한다.

$$A = A^T$$

예시

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

- 주대각선을 기준으로 좌우가 대칭
- 내적, 이차형식, 공분산 행렬 등에서 매우 중요

[4] 복소 행렬과 에르미트 행렬(Hermitian Matrix)

[4-1] 복소수 행렬 예시

$$A = \begin{bmatrix} 1 & 1+j \\ 1-j & 2 \end{bmatrix}$$

여기서 $j = \sqrt{-1}$

[4-2] 켤레 전치(Conjugate Transpose)

$$A = \begin{bmatrix} 1 & 1+j \\ 1-j & 2 \end{bmatrix} \Rightarrow A^* = \begin{bmatrix} 1 & 1-j \\ 1+j & 2 \end{bmatrix}$$

- A^* : 각 원소의 복소수 켤레
- $A^H = (A^*)^T$: 에르미트 전치

[4-3] 에르미트 행렬(Hermitian Matrix)

$$A^H = A$$

즉,

- 전치 + 복소 켤레를 취해도 자기자신
- 실수 행렬에서의 대칭 행렬에 대응되는 개념

내적(Dot Product, Scalar Product)

[1] 내적의 계산 정의

두 벡터

$$a = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, b = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

에 대해 내적은 다음과 같이 계산된다.

$$a \cdot b = (1 \times 5) + (3 \times 1) = 8$$

행렬 표현으로 쓰면,

$$a^T b = [1 \ 3] \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

또한 내적은 교환 가능하므로

$$a^T b = b^T a$$

가 성립한다.

[2] 기하학적 정의: 각도와 관계

내적은 단순한 계산 규칙이 아니라, 두 벡터 사이의 기하학적 관계를 담고 있다.

$$a^T b = \|a\| \|b\| \cos \theta$$

여기서

- θ 는 벡터 a 와 b 사이의 각도
- $\cos \theta$ 는 두 벡터가 얼마나 같은 방향을 향하고 있는지를 나타낸다.

즉,

- 같은 방향 $\rightarrow \cos \theta = 1$
- 수직 $\rightarrow \cos \theta = 0$
- 반대 방향 $\rightarrow \cos \theta = -1$

[3] 정사영(Projection)으로서의 내적 해석

위 식을 다음과 같이 묶어서 보면 더 분명해진다.

$$a^T b = (\|a\| \cos \theta) \|b\|$$

여기서

$$\|a\| \cos \theta$$

는 벡터 a 를 벡터 b 방향으로 정사영했을 때의 길이를 의미한다.

즉 내적은 벡터 a 가 벡터 b 방향으로 얼마나 투영되는가를 수치로 나타낸 결과이다.
그래서 내적에는 정사영의 의미가 내포되어 있다고 말한다.

[4] 자기 자신과의 내적: 벡터의 크기

벡터 a 를 자기 자신과 내적하면

$$a^T a = \|a\| \|a\| \cos \theta$$

이때 두 벡터는 완전히 같은 방향이므로

$$\cos \theta = 1$$

따라서,

$$a^T a = \|a\|^2$$

이 식을 정리하면,

$$\|a\| = \sqrt{a^T a}$$

단위 벡터(Unit Vector)와 정사영의 의미

[1] 단위 벡터(Unit Vector)

단위 벡터란 크기가 1 인 벡터를 의미한다.

임의의 벡터 a 가 있을 때, 그 방향은 유지하면서 크기만 1 로 만든 벡터는 다음과 같이 정의된다.

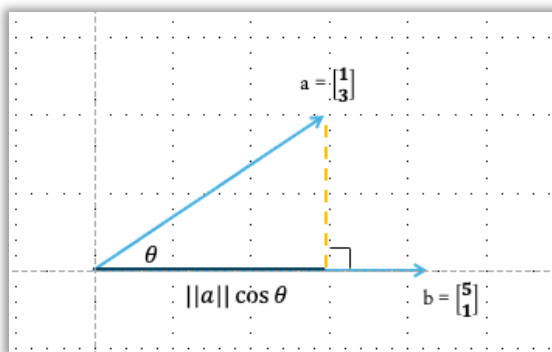
$$\hat{a} = \frac{a}{\sqrt{a^T a}} = \frac{a}{\|a\|}$$

이 연산을 정규화(normalization)라고 한다.

- 분자 a : 방향 정보
- 분모 $\|a\|$: 크기 정보

자기 자신의 크기로 나누면 방향은 그대로 유지되고, 크기만 1 이 된다.

[2] 벡터 a 를 벡터 b 방향으로 정사영하기



[2-1] 크기만 먼저 생각하기

내적의 식에서 $\|b\|$ 로 나누면,

$$\|a\| \cos \theta = \frac{a^T b}{\|b\|}$$

이는 벡터 a 가 벡터 b 방향으로 얼마나 길게 투영되는지를 의미하는 스칼라 값이다.

[2-2] 방향까지 포함시키기

하지만 “길이”가 아니라 벡터를 원하기에 방향 정보를 다시 곱해줘야 한다.

벡터 b 의 단위 벡터는

$$\hat{b} = \frac{b}{\sqrt{b^T b}} = \frac{b}{\|b\|}$$

따라서 정사영 벡터는

$$\left(\frac{a^T b}{\|b\|^2}\right) \hat{b} = \frac{a^T b}{\sqrt{b^T b}} \times \frac{b}{\sqrt{b^T b}} = \frac{a^T b}{b^T b} b$$

이 식이 벡터 a 를 벡터 b 위로 정사영한 결과이다.

[3] 실제 계산 예시

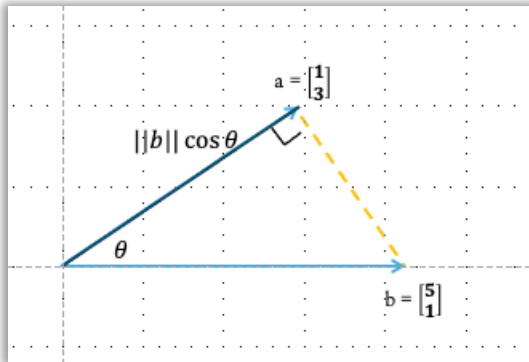
$$a = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, b = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

- $a^T b = 8$
- $b^T b = 26$

따라서,

$$\frac{a^T b}{b^T b} b = \frac{8}{26} \begin{bmatrix} 5 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{40}{26} \\ \frac{8}{26} \end{bmatrix}$$

[4] 반대로: 벡터 b 를 벡터 a 방향으로 정사영



이번에는 기준 방향을 a 로 바꾸면,

$$proj_a(b) = \frac{a^T b}{a^T a} a$$

계산하면,

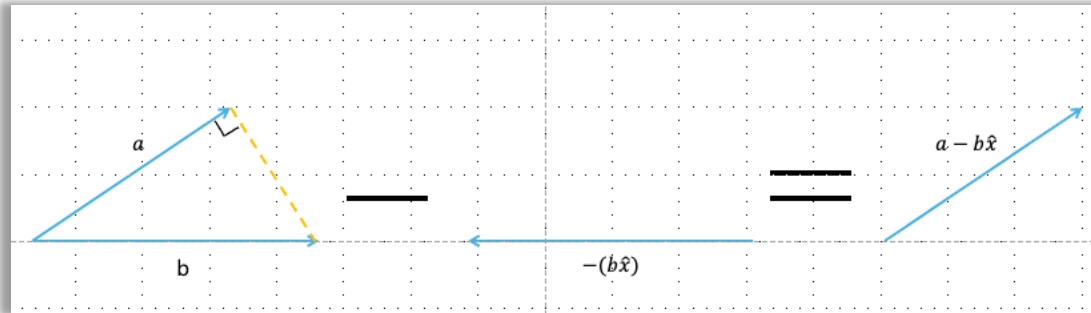
- $a^T a = 10$

$$\frac{a^T b}{a^T a} a = \frac{8}{10} \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} \frac{8}{10} \\ \frac{24}{10} \\ \frac{10}{10} \end{bmatrix}$$

정사영은 항상 기준 벡터가 무엇이냐에 따라 달라진다.

[5] 변의: 정사영 계수의 의미(최소 거리 관점)

다음 식을 보면



$$\hat{x} = \frac{a^T b}{b^T b}$$

이 값은 단순한 계산 결과가 아니라, a와 b의 방향 차이를 최소로 만드는 스칼라 라는 의미를 가진다.

- $b\hat{x}$ 는 b 방향으로 스칼라만큼 늘린 벡터
- $a - b\hat{x}$ 는 a에서 그만큼 뺀 오차 벡터
- 이 오차가 b와 수직일 때, 가장 가까운 점이 된다.

즉,

$$(a \cdot \hat{x}) \perp (b \cdot \hat{x}) = (a - b\hat{x})^T b\hat{x} = 0$$

전개하면,

$$a^T b - b^T b\hat{x} = 0$$

따라서,

$$\hat{x} = \frac{a^T b}{b^T b}$$

이 값은 정사영의 계수이자, 최소제곱 해의 핵심 값이다.

벡터의 크기와 Norm 의 개념

[1] Norm 이란 무엇인가

Norm 은 벡터의 크기 또는 길이를 수치로 정의하는 함수이다.

우리가 흔히 사용하는 벡터의 길이 개념은 사실 여러 가지 norm 중 하나에 불과하다.

지금까지 우리가 자연스럽게 사용해온 벡터의 크기는 2-norm(l_2 - norm)이다.

[2] 2-norm(l_2 - norm): 우리가 가장 익숙한 거리

벡터

$$x = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

의 2-norm 은 다음과 같이 정의된다.

$$\|x\|_2 = \sqrt{1^2 + 3^2} = \sqrt{x^T x}$$

이를 지수 형태로 쓰면,

$$\sqrt{1^2 + 3^2} = (1^2 + 3^2)^{\frac{1}{2}}$$

여기서 중요한 관찰은

- 제곱 $\rightarrow$ 합 $\rightarrow \frac{1}{2}$ 승

이 구조 때문에 이 norm 을 2-norm 이라고 부른다.

[2-1] 2-norm 과 거리의 관계

두 벡터

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, y = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

가 있을 때, 두 벡터 사이의 거리는

$$\|x - y\|_2$$

로 정의된다.

$$x - y = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \Rightarrow \|x - y\|_2 = \sqrt{(-1)^2 + 1^2} = \sqrt{2}$$

이는 고등학교에서 배운 두 점 사이의 거리 공식과 정확히 같은 개념이다.

즉, 2-norm 은 유클리드 거리(Euclidean distance)를 측정한다.

[3] 1-norm(l_1 - **norm**): 이동량의 합
이번에는 제곱이 아니라 1 승을 사용한다.

$$\|x\|_1 = |x_1| + |x_2| + \dots$$

왜 절대값이 필요한가?

- 벡터의 크기는 음수가 될 수 없기 때문
- 방향이 아니라 “총량”을 측정하기 때문

예를 들어,

$$b = \begin{bmatrix} 1 \\ 2 \\ -3 \end{bmatrix}$$

$$\|b\|_1 = |1| + |2| + |-3| = 6$$

1-norm 은 “얼마나 많이 움직였는가”의 총합을 의미한다.

- 격자(grid) 위에서의 이동 거리
- Manhattan distance
- 희소성을 강조하는 모델에서 자주 사용

[4] p-norm: Norm 의 일반화

2-norm 과 1-norm 은 사실 하나의 큰 틀 안에 있다.

이를 p-norm 이라 부른다.

$$\|x\|_p \triangleq \left(\sum |x_i|^p \right)^{\frac{1}{p}} \quad (p \geq 1)$$

- $p = 1 \rightarrow l_1$ - norm
- $p = 2 \rightarrow l_2$ - norm
- $p = 3, 4, \dots \rightarrow$ 다른 형태의 크기 정의

즉, norm 은 “무엇을 크기라고 볼 것인가”에 대한 선택이다.

[5] Infinity Norm($l_\infty - norm$): 가장 큰 성분만 본다

이제 $p \rightarrow \infty$ 로 보낸다.

$$\|x\|_\infty \triangleq \max_i |x_i|$$

직관적으로 보면,

$$(|x_1|^\infty + |x_2|^\infty + \dots)^{\frac{1}{\infty}}$$

에서 가장 큰 값만 살아남고, 나머지는 무시된다.

예를 들어,

$$x = \begin{bmatrix} 1 \\ -4 \\ 2 \end{bmatrix}$$

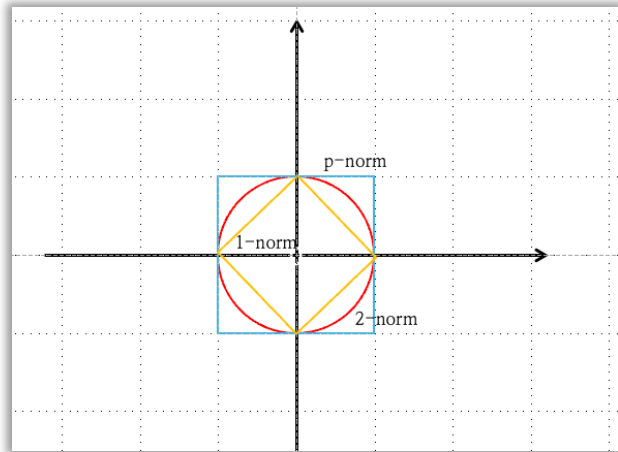
$$\|x\|_\infty = 4$$

주로 infinity norm 은 최악의 경우, 최대 오차, 한계 값을 측정할 때 사용된다.

[6] Norm 들의 의미 비교

Norm	의미	직관
$l_1 - norm$	총 이동량	얼마나 많이 움직였나
$l_2 - norm$	직선 거리	실제 거리
$l_\infty - norm$	최대 성분	가장 큰 영향

[7] Norm ball 로 이해하는 벡터 크기의 차이



이 그림은 원점(0,0)을 중심으로

$$\|x\|_p = 1$$

을 만족하는 점들의 집합, 즉 unit norm ball 을 비교한 것이다.

- 같은 “크기 = 1”이라도 어떤 norm 을 쓰느냐에 따라 허용되는 벡터의 모양이 달라진다.

[7-1] l_2 - **norm**(2-norm): 원 (노란색)

정의

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2}$$

그림에서 의미

- 완벽한 원(circle)
- 모든 방향을 동일하게 취급
- 회전에 대한 불변

직관

- 진짜 거리: 유클리드 공간에서의 자연스러운 길이
- 물리, 기하, 대부분의 머신러닝 기본 거리에서 기본값으로 사용된다.

[7-2] l_1 - **norm**(1-norm): 마름모(빨간색)ff

정의

$$\|x\|_1 = |x_1| + |x_2|$$

그림에서의 의미

- 마름모(diamond)
- 축 방향에 꼭짓점이 있음

직관

- 총 이동량의 합: 한 축씩 이동하는 비용
- 희소성(sparsity), feature selection, Lasso 와 직결된다.

꼭짓점이 축 위에 있다는 것이 중요하다. 즉, 한 성분만 크고 나머지는 0 인 벡터를 선호한다.

[7-3] l_∞ - **norm**(무한 노름): 정사각형(파란색)

정의

$$\|x\|_\infty \triangleq \max(|x_1|, |x_2|)$$

그림에서 의미

- 정사각형
- 가장 큰 성분 하나만 기준

직관

- 최악의 경우만 본다: 최대 허용 오차
- 제어 시스템, 안정성 분석, 로봇/자율주행 안전 제한

벡터의 행렬의 곱: 연립방정식에서 공간 변환까지

[1] 연립일차방정식과 행렬 표현

다음과 같은 연립일차방정식을 생각한다.

$$\begin{cases} x + 2y = 4 \\ 2x + 5y = 9 \end{cases}$$

이 식은 행렬과 벡터를 이용해 다음과 같이 표현할 수 있다.

$$\begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 4 \\ 9 \end{bmatrix}$$

이 표현은 단순한 표기법의 변화가 아니라, 여러 방정식을 하나의 구조로 묶어 공간적으로 해석할 수 있게 해준다는 점에서 중요하다.

[2] 여러 벡터를 동시에 곱하는 경우

이제 하나의 벡터가 아니라, 여러 벡터를 동시에 입력으로 넣어본다.

$$\begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix} = \begin{bmatrix} 4 & 3 \\ 9 & 7 \end{bmatrix}$$

이는 사실 다음 두 계산을 동시에 수행한 것이다.

$$\begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} 4 \\ 9 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}$$

즉, 행렬×행렬 = 여러 벡터에 동일한 선형 변환을 동시에 적용 이라는 의미를 가진다.

[3] 행렬 곱의 크기 규칙

행렬 곱이 가능하기 위한 조건과 결과의 크기는 다음과 같다.

[3-1] 곱셈 가능 조건

$$(m \times k) \cdot (k \times n)$$

안쪽 차원 k 가 반드시 같아야 한다.

[3-2] 결과 행렬의 크기

$$(m \times k) \cdot (k \times n) = (m \times n)$$

즉,

- | |
|--|
| <ul style="list-style-type: none">- 왼쪽 행렬의 행 개수- 오른쪽 행렬의 열 개수 |
|--|

가 결과 행렬의 크기를 결정한다.

[3-3] 순서의 중요성

행렬 곱은 일반적으로

$$AB \neq BA$$

즉, 교환법칙이 성립하지 않는다.

예를 들어,

$$(4 \times 3)(3 \times 5)(5 \times 2)$$

는 계산 가능하지만 순서를 바꾸면 아예 곱이 정의되지 않을 수도 있다.

[4] 행렬 곱을 이해하는 네 가지 관점

행렬 곱은 단순 계산이 아니라, 어떤 관점으로 보느냐에 따라 의미가 달라진다.

들어가기 앞서, 보통 벡터를 나타낼 때 column vector 로 나타내므로

$$A = [a_1 \ a_2 \ a_3]$$

가로로 쌓인 벡터를 표현하기 위해서 Transpose 로 표현한다.

$$A = \begin{bmatrix} a_1^T \\ a_2^T \\ a_3^T \end{bmatrix}$$

[4-1] 내적(dot product) 관점

행렬 곱의 기본 원리는 행과 열의 내적이다.

$$(AB)_{ij} = (A \text{의 } i \text{ 번째 행}) \cdot (B \text{의 } j \text{ 번째 열})$$

즉,

- 한 원소는 항상 벡터 내적의 결과
- 행렬 곱은 내적을 격자로 확장하는 것

이 관점은 계산 규칙을 이해하는데 가장 직접적이다.

$$AB = \begin{bmatrix} a_1^T \\ a_2^T \\ a_3^T \end{bmatrix} [b_1 \ b_2 \ b_3] = \begin{bmatrix} a_1^T b_1 & a_1^T b_2 & a_1^T b_3 \\ a_2^T b_1 & a_2^T b_2 & a_2^T b_3 \\ a_3^T b_1 & a_3^T b_2 & a_3^T b_3 \end{bmatrix}$$

[4-2] Rank-1 행렬의 합으로 보기

행렬 곱은 다음과 같이 분해해서 볼 수 있다.

$$AB = \sum_k (A \text{의 } k \text{ 번째 열})(B \text{의 } k \text{ 번째 행})$$

각 항은

- 열벡터×행벡터
- 즉, rank-1 행렬

따라서, 행렬 곱은 여러 개의 rank-1 행렬을 더한 결과 라는 해석이 가능하다.

이 관점은 SVD, 저랭크 근사, 머신러닝에서 매우 중요하다.

[4-3] Column space 관점(입력 관점)

$$AB = [Ab_1 \ Ab_2 \ \dots]$$

즉,

- B 의 각 열벡터가
- A 에 의해 변환된 결과를
- 열 단위로 모아 놓은 것

행렬 A 는 입력 벡터를 새로운 공간으로 보내는 변환자

[4-4] Row space 관점(출력 관점)

반대로,

- A 의 각 행은
- B 의 column space 를 어떤 방향으로 측정하는 기준

즉, 출력은

- 행 공간(row space)에 의해 결정된다.

이 관점은 다음으로 이어진다.

- 선형 분류기
- 가중치 해석
- feature 중요도 분석

선형 결합(Linear combination)

선형 결합이란 여러 벡터에 각각 상수를 곱한 뒤 더해서 새로운 벡터를 말하는 것을 의미한다.

즉, 어떤 벡터가 다른 벡터들의 선형 결합으로 표현될 수 있다면, 그 벡터는 기존 벡터들이 만들어내는 공간 안에 존재한다고 말할 수 있다.

예를 들어 벡터 v_1, v_2 가 있을 때

$av_1 + bv_2$ (a, b 는 상수) 형태로 만들어지는 모든 벡터들이 선형 결합이다.

선형 결합은 벡터 공간, 기저, 차원 개념의 출발점이며 머신러닝, 그래픽스, 최적화 문제의 핵심 개념이다.

정사각 행렬(Square Matrix)

정사각 행렬은 행(row)과 열(column)의 개수가 같은 행렬을 말한다.

예를 들어 2×2 , 3×3 행렬이 이에 해당한다.

정사각 행렬은 다음과 같은 중요한 성질을 가진다.

- 역행렬이 존재할 수 있다.
- 행렬식(determinant)을 정의할 수 있다
- 고유값(eigenvalue), 고유벡터(eigenvector)를 가질 수 있다

따라서 선형대수에서 가장 중심적인 행렬 형태라고 할 수 있다.

단위 행렬(Identity Matrix)

단위 행렬(Identity Matrix)은 대각선 원소는 1, 나머지는 모두 0인 정사각 행렬이다.

단위 행렬의 가장 중요한 특징은

어떤 행렬 A 에 곱해도 A 가 변하지 않는다는 점이다.

즉,

- $A \times I = A$
- $I \times A = A$

이 성질 때문에 단위 행렬은 숫자에서의 1과 같은 역할을 한다.

역행렬(Inverse Matrix)

역행렬이란 어떤 행렬 A 에 곱했을 때 단위 행렬이 되는 행렬을 의미한다.

즉,

$$- A \times A^{-1} = I$$

역행렬은 모든 정사각 행렬에 존재하지는 않으며,
행렬식이 0 이 아닌 경우에만 존재한다.

역행렬은 연립방정식 풀이, 좌표 변환 복원, 머신러닝 계산에서 매우 중요하다.

대각 행렬(Diagonal Matrix)

대각행렬은 대각선 성분을 제외한 모든 원소가 0 인 정사각 행렬이다.

예를 들어,

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

대각 행렬은 연산이 매우 간단하다.

- 곱셈 시 각 축 방향으로 독립적인 스케일 조절
- 역행렬 계산이 쉽다(각 대각 원소의 역수)

그래서 수치 해석과 선형 변환에서 자주 사용된다.

직사각 대각 행렬(Rectangular Diagonal Matrix)

직사각 대각 행렬은 행과 열의 개수가 다른 행렬이지만
대각선 위치에만 값이 있고 나머지는 0 인 행렬이다.

주로 특이값 분해(SVD)에서 등장하며,
차원이 다른 공간 사이의 변환을 표현할 때 사용된다.

정사각 대각 행렬의 개념을 직사각 형태로 확장한 것이라고 이해하면 된다.

직교 행렬(Orthogonal Matrix)

직교 행렬은 열 벡터(또는 행 벡터)들이 서로 직교하며 길이가 1 인 행렬이다.

가장 중요한 성질은 다음과 같다.

- 역행렬 = 전치 행렬
- $A^{-1} = A^T$

직교 행렬은 다음과 같은 특징을 가진다.

- 길이를 보존한다
- 각도를 보존한다
- 회전 방향을 표현한다

그래서 컴퓨터 그래픽스, 로봇공학, 물리 시뮬레이션에서 매우 중요하다.

Rank(랭크)

[1] Rank 의 정의

Rank 란 행렬이 가지는 서로 독립적인(independent) column 의 개수이다.

$$\text{rank}(A) = \text{number of linearly independent column of } A$$

이는 다음과 완전히 동일한 의미를 가진다.

- column space 의 차원(dimension)
- row space 의 차원

즉,

$$\text{rank}(A) = \dim(\text{Col}(A)) = \dim(\text{Row}(A))$$

[2] 중요한 성질

[2-1] Independent column 수 = Independent row 수

행렬 A 에서

$$\text{independent columns 수} = \text{independent rows 수}$$

항상 같다.

[2-2] 전치해도 rank 는 변하지 않는다.

$$\text{rank}(A) = \text{rank}(A^T)$$

- A 의 column space $\leftrightarrow A^T$ 의 row space
- 차원이 같기 때문

[3] 예제

[3-1]

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \end{bmatrix}$$

- 두 번째 행이 모두 0
- 모든 column 이 서로 같은 방향(선형 종속)
- independent 한 column 은 1 개

$$\text{rank}(A) = 1$$

[3-2]

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

- 첫 번째 column 과 두 번째 column 은 서로 독립
- 세 번째 column 은 앞 두 column 의 합
- independent 한 column 은 2 개

$$\text{rank}(A) = 2$$

[4] Rank 관련 용어 정리

행렬 $A \in \mathbb{R}^{m \times n}$ 일 때:

[4-1] Full Column Rank

$$\text{rank}(A) = n$$

- 모든 column 이 선형 독립
- column space 의 차원이 최대
- $m \geq n$ 이어야 가능
- $A^T A$ 가 invertible

[4-2] Full Row Rank

$$\text{rank}(A) = m$$

- 모든 row 가 선형 독립
- row space 의 차원이 최대
- $n \geq m$ 이어야 가능
- AA^T 가 invertible

[4-3] Full Rank

$$\text{rank}(A) = \min(m, n)$$

- 가능한 최대 rank
- 정사각행렬에서는

$$\text{rank}(A) = n \Leftrightarrow A \text{ is invertible}$$

[4-4] Rank-deficient(랭크 결손)

$$\text{rank}(A) < \min(m, n)$$

- column 또는 row 중 일부가 선형 종속
- 정보 손실 존재
- 해가 유일하지 않거나 아예 없을 수 있음

머신러닝에서

- 데이터 중복
- feature redundancy
- overparameterization

Null space : $Ax = 0$ 을 만족하는 x 의 집합

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$
$$Ax = x_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} + x_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

을 만족시키고 싶을 때